Curso de Introducción a R Shiny

BIODAMA
Bioinformatics and Data Management

BY NC ND

GOBIERNO DE ESPAÑA
MINISTERIO DE CIENCIA, INNOVACIÓN Y UNIVERSIDADES
MINISTERIO DE SANIDAD
Instituto de Salud Carlos III
Ciencia y Salud 1986-2026

2. Introducción a R Shiny y sus fundamentos

2.1 ¿Qué es R Shiny?

- Shiny es un package de R para crear aplicaciones interactivas con estructura 'cliente-servidor'.
- Ha sido elaborado por los creadores de RStudio. (Joe Chen).
- Su base es R. Con conocimientos sólo de R, se pueden desarrollar aplicaciones web sin necesidad de conocimientos de lenguajes propios del desarrollo web.
- Aunque no es necesario, permite incorporar código de estos lenguajes de programación web, en parte cliente (HTML, JavaScript, CSS …), para obtener una mayor capacidad de personalización en las aplicaciones.

2.2 Estructura básica de una aplicación

- En un único fichero (app.R):

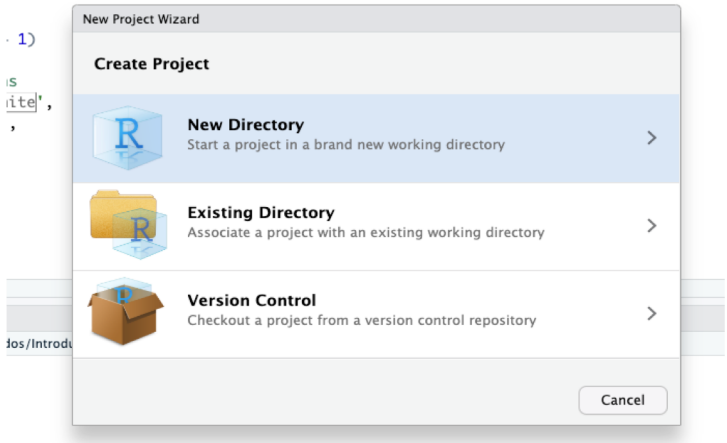
```
1  library(shiny)
2
3  ### Parte cliente
4  ui <- fluidPage(
5
6            "1ª Aplicación Shiny"
7  )
8  ### Parte servidor
9  server <- function(input, output) {
10
11 }
12
13 # Ejecutar la app ----
14 shinyApp(ui = ui, server = server)
15
```

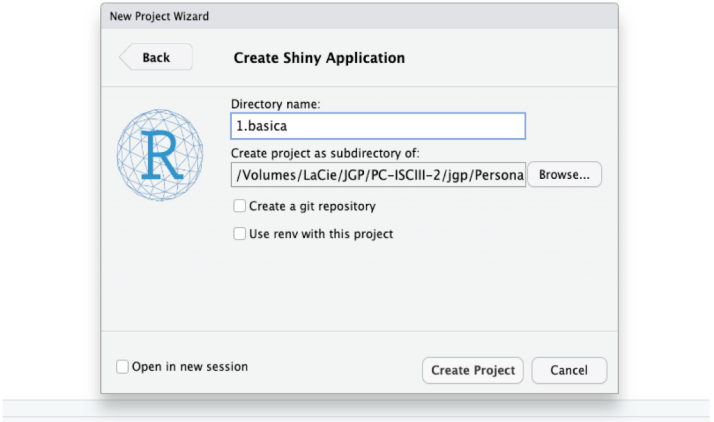
- Estructurado en varios ficheros:



2. Introducción a R Shiny y sus fundamentos

- Desde las opciones que nos proporción RStudio (File -> New Project…)

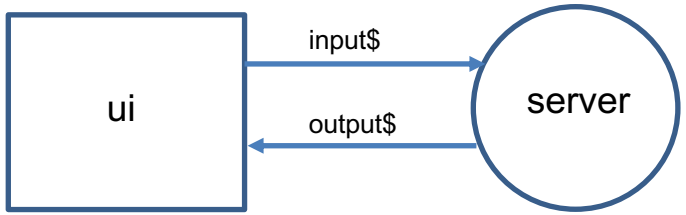




2. Introducción a R Shiny y sus fundamentos

2.3 Ciclo de reactividad

- La comunicación entre el cliente 'ui' y el servidor 'server', se realiza:



- Shiny recalcula automáticamente todo le que se vea afectado por un cambio en algún 'input'.
- Los pasos son los siguientes:
  - Usuario modifica algún input
  - Shiny recalcula las expresiones reactivas necesarias
  - Shiny actualiza los outputs afectados
  - Shiny refresca la interfaz (ui)

? En la aplicación '1.basica', ¿qué input afecta al ciclo de reactividad?

2. Introducción a R Shiny y sus fundamentos

- Los **principales componentes** del sistema reactivo de shiny son:
  - 'input$': input reactivo
  - 'reactive()': expresión reactiva (dependiente de algún input reactivo, pero no necesario en ese mismo instante)
  - 'observe()' y 'observeEvent()': observadores
  - 'renderPlot()', 'renderTable()' … : salidas reactivas

- Las variables reactivas deben tratarse **siempre** en un entorno reactivo:
  - Correcto:

```
server <- function(input, output) {

    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui.R
        x    <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })

    observeEvent(input$bins,{
      print(input$bins)
    })
}
```

2. Introducción a R Shiny y sus fundamentos

– Erróneo:

```
server <- function(input, output) {

    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui.R
        x    <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })

    print(input$bins)
}
```

- Se puede desactivar la reactividad de una determinada variable, en alguna parte del código:

```
server <- function(input, output) {

    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui.R
        x    <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })

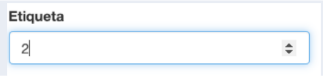
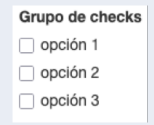
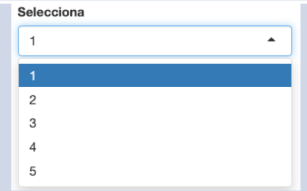
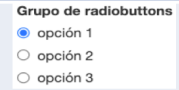
    print(isolate(input$bins))
}
```

Run the Shiny application

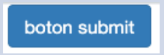
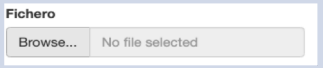
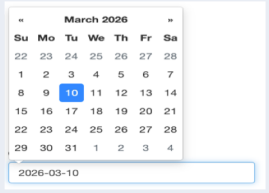
3. Elementos de entrada/salida

3.1 Entrada

- Las entradas se sitúan en la parte cliente (ui). Esta es la lista de funciones que proporciona Shiny para crear elementos de entrada (widgets):

Función	Descripción	Aspecto
textInput	Campo de entrada de texto	
numericInput	Campo de entrada de números	
checkboxInput	Checkbox	
checkboxGroupInput	Grupo de checkbox *(no excluyentes)*	
selectInput	Combo box	
radioButtons	Conjunto de checks *(excluyentes)*	

3. Elementos de entrada/salida

Función	Descripción	Aspecto
actionButton	Botón	
actionLink	Enlace. *Similar a actionButton. Cambia el aspecto*.	
(*) submitButton (desaconsejado)	Botón de enviar.	
sliderInput	Rango de valores.	
fileInput	Control para subir fichero.	
dateInput	Entrada de fecha.	
helpText	Texto de ayuda.	

(*) Inhabilita la reactividad de toda la aplicación, hasta que el botón se pulsa. Está en desuso.

3. Elementos de entrada/salida

- Actualización de 'inputs' desde la parte servidor (server): los principales componentes inputs, tienen una función que permite actualizar su definición desde el 'server'. Se nombran como update…

```
131  # Actualización de 'inputs' desde el 'servidor'
132  updateSelectInput(session = session, inputId = "id_s", label = "Selecciona", choices = c("a","b","c","d"), selected = "b")
133
```

- Shiny permite incluir etiquetas 'html', de forma simple mediante las funciones de tags. Cada etiqueta html tiene su correspondiente función shiny. Por ejemplo:

  tags$a() ***equivale a*** <a>
  tags$a("CNE", href="https://cne.isciii.es/") CNE

- Podemos omitir el prefijo 'tags$' en muchas de estas funciones al formar parte del propio paquete shiny:

  tags$h2() o h2 ***equivale a*** <h2>
  h2("Texto Head 2") <h2>Texto Head 2</h2>

- Se puede ver la lista completa de tags en: https://shiny.posit.co/r/articles/build/tag-glossary/

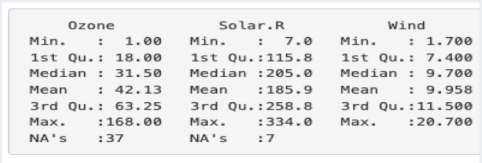
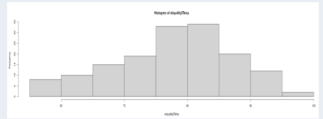
3. Elementos de entrada/salida

- Además de los controles básicos que hemos visto, existen paquetes de controles más vistosos y que añaden alguna funcionalidad extra: https://dreamrs.github.io/shinyWidgets/index.html

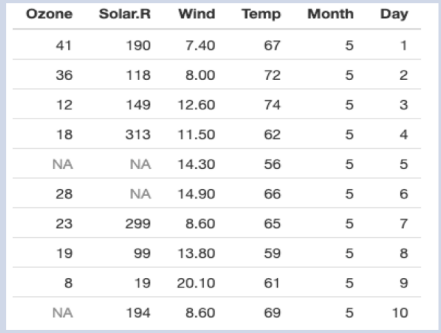
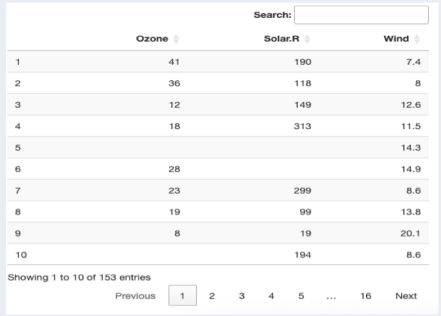
3. Elementos de entrada/salida

3.2 Salida

- Las salidas se sitúan en la parte cliente (ui), pero se generan en la parte servidor (server). Esta es la lista de funciones que proporciona Shiny para crear elementos de salida:

Función	Descripción	Aspecto
textOutput	Campo de texto *(interpreta html)*	Ejemplo de textOutput
verbatimTextOutput	Campo de texto, preservando el formato *(como la salida por consola de R)*	
imageOutput	Imagen estática	
plotOutput	Gráfico dinámico	

3. Elementos de entrada/salida

Función	Descripción	Aspecto
tableOutput	Tabla	
dataTableOutput *(obsoleta)* DT::renderDT *(actualmente)*	Tabla. Se crea un contenedor, parametrizable.	
uiOutput	Insertar componentes UI de Shiny dinámicamente.	
htmlOutput	Inserta contenido html, dinámicamente.	

3. Elementos de entrada/salida

3.3 Diseño adaptativo (bootstrap)

- Bootstrap son librerías de código HTML, CSS y JavaScript, que nos proporcionan funcionalidad de diseño web auto-ajustable a los distintos tamaños de pantalla y dispositivos.
- Shiny contiene funciones, que nos dan esta funcionalidad directamente, sin necesidad de utilizar estos lenguajes mencionados. En general llevan el prefijo fluid:
- fluidPage(): página auto-ajustable, que contiene tablas (filas, columnas) y escala sus componentes para ocupar todo el ancho de pantalla. El número de columnas, siempre será 12, de igual ancho.
- fluidRow(): fila de una página auto-ajustable.
- column(): columnas de una fila auto-ajustable. No se nombran como fluid porque también pueden utilizarse en fixedPage().
- Estas funciones, nos van a permitir organizar nuestro contenido web (ui), de forma que se adapte según el tamaño de la pantalla donde se visualiza y no pierda la estructura.
- En la practica, esto se cumple dentro de ciertos límites, así en pantallas de PC de distintos tamaños, no suele haber problemas, pero a la hora de visualizarse en tamaños como dispositivos móviles, podemos encontrar contenido que no encaja bien visualmente o que aparece descolocado.

3. Elementos de entrada/salida

```
ui <- fluidPage(

  fluidRow(
    column(1,""),
    column(5, align="center",
           textInput("id_t", "Etiqueta", placeholder = "Rellenar con texto"),
           ),
    column(5, align="center",
           numericInput("id_n","Etiqueta", value = 2, min=0, max=10, step=1),
           ),
    column(1,"")
  ),

  fluidRow(
    column(1,""),
    column(5, align="center",
           checkboxInput("id_c", "un check"),
           ),
    column(5, align="center",
           selectInput("id_s", "Selecciona", choices = c(1:5)),
           ),
    column(1,"")
  )

)
```

3. Elementos de entrada/salida



Ejercicio:

Partiendo de la base de datos del paquete base de R ‘cars’, hacer una aplicación R Shiny, que:

1. Muestre una tabla con los valores de estos datos (cars)
2. Incluya un control que permita seleccionar los límites inferior y superior de velocidad, seleccionando inicialmente el menor valor más uno y el mayor valor menos uno.
3. El diseño de la aplicación debe ser adaptativo.

4. Interactividad y reactividad avanzada

4.1 Funciones clave

- Las principales funciones mediante las cuales podemos controlar la reactividad, en el servidor, son:
  - reactive(), observe(), observeEvent(), reactiveValues()

- Hay que tener en cuenta que una variable o expresión reactiva, se actualiza automáticamente cuando ésta se actualiza en el cliente (ui), o en el servidor (si la hemos declarado como reactiva), y además provoca que se actualicen otras expresiones dependientes de ella (todas las que la utilicen).
  - reactive(): Función que engloba expresiones o variables reactivas.
  - observe(): Crea un observador reactivo. Es decir, un 'listener' que se activa si se modifica el valor de alguna de las variables o expresiones reactivas que contenga.
  - observeEvent(): Similar a observe, pero en este caso, sólo se activa si varia la expresión o variable a evaluar (parámetro 'eventExpr').
  - reactiveValues(): Genera una lista, donde cada componente de la lista es un valor reactivo con comportamiento similar a un input del cliente (ui), en lo referente a reactividad. Es decir, si se modifica en el propio server algún elemento de la lista, se genera un evento reactivo. Existe otra función similar (reactiveVal), que crea un único elemento reactivo, en vez de una lista.

4. Interactividad y reactividad avanzada

- Hemos visto la reactividad generada principalmente desde los 'input', pero también existe reactividad en el sentido contrario, de forma que los 'output' se recalculan automáticamente cuando hay un cambio en los datos que se usan para su generación.

  Vimos varios ejemplos en el apartado '3.2. Salida':

  - renderPlot(), renderTable(), renderText() …

4. Interactividad y reactividad avanzada

4.2 Control de dependencias reactivas

- Características de las principales funciones reactivas:

	reactive()	***observe()***	***observeEvent()***	***reactiveValues()***
Devuelve un valor	✓	✗	✗	✓
Ejecuta un conjunto de instrucciones reactivas	✗	✓	✓	✗
Se ejecuta dependiendo del cambio de valor de un parámetro	✗	✗	✓	✗
Se ejecuta dependiendo del cambio de valor de alguna de las variables que contiene su código	✓	✓	✗	✗
Su modificación provoca que se ejecute código reactivo que lo utiliza	✗	✗	✗	✓

- En resumen:
- Almacenar estado mutable: reactiveValues() y reactiveVal()
- Cálculos dependientes: reactive()
- Control de efectos secundarios: observe() y observeEvent()

4. Interactividad y reactividad avanzada

- Además, debemos recordar la supresión de la reactividad mediante la función isolate():

```
39  # Ejemplo supresión de reactividad (isolate)
40  observe({
41    print(c("isolate - id1:", isolate(input$id1)))
42    print(c("id3:", input$id3))
43  })
```

- En este código, se ejecutarán todas las instrucciones contenidas en el 'observe', cuando el input 'id3' modifique su valor, pero no si el input 'id1' cambia, puesto que suprimimos su reactividad mediante la función 'isolate()'.

? En una aplicación web R Shiny, que necesita cargar un fichero de datos desde el navegador, ¿qué función reactiva utilizarías para almacenar los datos?

4. Interactividad y reactividad avanzada



Ejercicio:

Partiendo del ejercicio del apartado 3., modificar la aplicación R Shiny, de forma que cumpla:

1. Muestre una tabla con los valores de estos datos (cars).
2. Incluya un control que permita seleccionar los límites inferior y superior de velocidad, para que en función de esta selección se actualice la tabla mostrada en el punto 1.
3. Incluya un selectInput, con los valores de las variables de estos datos (cars), de forma que según la variable que se seleccione, se modifique el control del 'punto 2' para que muestre la etiqueta de dicha variable, así como el rango de posibles valores de esta variable seleccionada.

5. Gestión de datos y seguridad

5.1 Lectura de datos desde ficheros externos

- La lectura de los ficheros necesarios para incorporar datos en nuestra aplicación la haremos mediante la función *fileInput()*, en la parte cliente (ui) y capturaremos sus actualizaciones mediante reactividad en la parte servidor (*reactive(), observeEvent()*…)

```
5  datos <- reactive({
6
7    shiny::req(input$id_carga)
8    aux <- openxlsx::read.xlsx(xlsxFile=input$id_carga$datapath)
9    str(aux)
0
1    # Validación del formato del fichero
2    dup <- duplicated(names(aux))
3    shiny::validate(need(sum(dup)==0,paste("Hay variables duplicadas!:",names(aux)[dup])))
4
5    aux
6
7  })
```

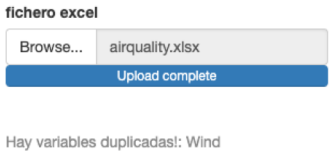
5. Gestión de datos y seguridad

5.2 Descarga de datos

- Si en vez de cargar datos en nuestra aplicación, necesitamos descargarlos, utilizaremos las funciones downloadButton() en 'ui' y la función downloadHandler() en la parte 'server'.

```
44    output$id_descarga <- downloadHandler(
45      filename = function() {
46        paste("curso_R_Shiny", ".xlsx", sep="")
47      },
48      content = function(file) {
49        sub_base <- airquality[ ,c(1:3)]
50        openxlsx::write.xlsx(sub_base, file)
51      }
52    )
```

- Las validaciones pueden hacerse de varias formas:
  - Más simples, con mensajes en pantalla.

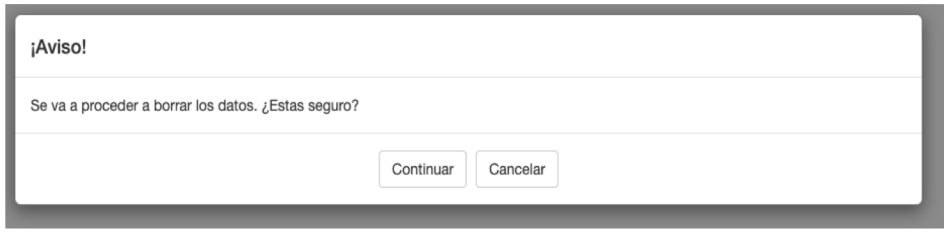


5. Gestión de datos y seguridad

– Mediante popups (ventanas emergentes) que muestren el mensaje.



– Existen otro tipo de ventanas emergentes que nos dan otra funcionalidad, son las ventanas modales. En este caso, se le da al usuario de la aplicación la posibilidad de elegir o rechazar una acción.



5. Gestión de datos y seguridad

5.3 Securización

A veces necesitaremos restringir el acceso a nuestra aplicación, para lo que deberemos implementar una página inicial de autenticación (login).
Shiny dispone de varios paquetes que nos proporcionan funciones para esta implementación:

library(shinymanager)
library(shinyauthr)

Vamos a utilizar 'shinymanager':
En el 'ui', usaremos la función 'secure_app' englobando todo el código del ui, mientras que en el 'server' pondremos la base de datos con los usuarios y contraseñas y la función secure_server() que se encarga de la validación de estos datos proporcionados por el usuario.

```
10  ui <- shinymanager::secure_app(
11    fluidPage(
12    |
13    )
14  )
15
```

5. Gestión de datos y seguridad

```
6  server <- function(input, output) {
7
8    # Definimos usuarios y contraseñas
9    credentials <- data.frame(
10     user = c("admin", "user"),
11     password = c("123", "abc"),
12     stringsAsFactors = FALSE
13   )
14
15   res_auth <- secure_server(check_credentials(credentials))
```

El 'data.frame' proporcionado a 'credentials', debe tener unos nombres de columna concretos (user, password…). Para mayor detalle, consultar la ayuda de la función check_credentials().

5. Gestión de datos y seguridad

Con lo visto hasta ahora, el desarrollador proporciona por código, los usuarios y contraseñas. Si necesitamos que la contraseña sea anónima, y el desarrollador no la introduzca manualmente, habría que implementar una página de registro, y guardar en una base de datos encriptada el valor de la contraseña junto con el usuario registrado. Normalmente, se utiliza una base de datos SQL, pero hay más opciones.

```
######### En el proceso de registro ##########
credentials <- data.frame(
  user = c(input$usuario),
  password = c(scrypt::hashPassword(input$contrasena)),
  is_hashed_password = TRUE,
  stringsAsFactors = FALSE
)

saveRDS(credentials, "usuarios.rds")

######### Al iniciar la aplicación ##########
credentials <- readRDS("usuarios.rds")

res_auth <- shinymanager::secure_server(
  check_credentials = shinymanager::check_credentials(credentials)
)
```

5. Gestión de datos y seguridad

5.4 Ámbito de las variables.

- El ámbito de una variable, es el alcance de ésta dentro del código, es decir, en que parte del código la tenemos disponible cuando necesitamos usarla.
- De forma general, en shiny, podemos distinguir estos ámbitos:
  - *Global*: se define en un archivo a parte de la aplicación pero en el mismo directorio, llamado 'global.R'. Las variables y código incluido aquí, son compartidas por todos los usuarios de la aplicación y se ejecuta una sola vez al iniciar ésta.

    Normalmente, se usa para variables constantes, cuyo valor no va variar durante la ejecución, o funciones usadas en varias partes del código.

    Es importante tener en cuenta que si vamos a publicar nuestra aplicación, de forma que puede ser usada por varios usuarios, éstos crearan distintas sesiones con acceso al entorno global, de forma que si una sesión modifica alguna variable global, ésta queda modificada para los demás usuarios (sesiones).

  - *De sesión*: básicamente afecta al 'server'. Cada usuario que entra en la aplicación genera una copia de las variables que se utilizan en el 'server', que son propias de esa sesión.

5. Gestión de datos y seguridad

- *Locales*: este ámbito se refiere a las partes de códigos aisladas, esto es, 'observe', 'render', 'function' … Las variables definidas ahí, sólo son accesibles en dichas partes del código.

? En el siguiente código, la variable 'aux', ¿qué ámbito tiene?

```
20  # upload de fichero de datos
21  datos <- reactive({
22
23    shiny::req(input$id_carga)
24    aux <- openxlsx::read.xlsx(xlsxFile=input$id_carga$datapath)
25    str(aux)
26
27    # Validación del formato del fichero
28    dup <- duplicated(names(aux))
29    shiny::validate(need(sum(dup)==0,paste("Hay variables duplicadas!:",names(aux)[dup])))
30
31    aux
32
33  })
```

5. Gestión de datos y seguridad



Ejercicio:

- Hacer con R Shiny, una aplicación que proporcione la opción de leer un fichero con datos, y muestre un gráfico a partir de dicho fichero. Además debe incluir 2 combos (selectInput), con las variables de los datos del fichero, para poder parametrizar el gráfico.
- Por último, incorporará un botón de descarga de los datos del fichero. Se debe utilizar el fichero 'global.R'.

6. Visualización de datos

6.1 Tablas interactivas.

- Existen varios paquetes de R, que nos dan la opción de mostrar datos en tablas interactivas, que facilitan al usuario la visualización, búsqueda y filtrado de estos datos, además de ser mejor estéticamente. El principal y más utilizado es 'DT', pero hay otros como 'reactable'.
- DT: este paquete está basado en la librería 'DataTables' de JavaScript https://datatables.net/, por lo que disponemos de su funcionalidad dentro de R. El paquete de R, tiene definidos sus propios parámetros que evitan tener que introducir código JavaScript, pero a su vez permite utilizarlo para una mayor potencia y personalización de las opciones.
- Nos posibilita las siguientes funcionalidades:
  - Búsqueda
  - Ordenación
  - Paginación
  - Filtros por columna
  - Edición de celdas
  - Exportación (csv. xlsx…)

6. Visualización de datos

- Una implementación básica con DT:

```
8     output$id_tablaDT <- DT::renderDT( airquality )
```

- Nos ofrece muchas de estas funcionalidades (búsqueda, ordenación, paginación):

Show 10 entries Search:

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5			14.3	56	5	5
6	28		14.9	66	5	6
7	23	299	8.6	65	5	7
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10		194	8.6	69	5	10

Showing 1 to 10 of 153 entries Previous 1 2 3 4 5 … 16 Next

6. Visualización de datos

- *Para filtrado*, utilizaremos la función 'datatable':

```
10      # Con filtrado
11      output$id_tablaDT <- DT::renderDT( datatable(airquality, filter = "top") )
```

Show 10 entries Search:

	Ozone	Solar.R	Wind	Temp	Month	Day
	All	All	All	All	All	All
1	41	190	7.4	67	5	1
2	36	118	8	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5			14.3	56	5	5
6	28		14.9	66	5	6
7	23	299	8.6	65	5	7
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10		194	8.6	69	5	10

Showing 1 to 10 of 153 entries Previous 1 2 3 4 5 … 16 Next

6. Visualización de datos

- *Para edición*, tenemos otro parámetro de la función 'datatable':

```
13      # Editable
14      output$id_tablaDT <- DT::renderDT( datatable(airquality, editable = TRUE) )
```

Show 10 entries

Search:

	Ozone	Solar.R	Wind	Temp	Month	Day
1	50	190	7.4	67	5	1
2	36	118	8	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5			14.3	56	5	5
6	28		14.9	66	5	6
7	23	299	8.6	65	5	7
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10		194	8.6	69	5	10

Showing 1 to 10 of 153 entries

Previous 1 2 3 4 5 … 16 Next

6. Visualización de datos

- Esto, no modifica los datos originales, sólo la tabla visualizada. Podemos capturar el evento de modificación de un valor, usando como referencia 'identificador_de_la_tabla_cell_edit' :

```
16      # Capturar modificaciones de la tabla
17      observeEvent(input$id_tablaDT_cell_edit, {
18        celda <- input$id_tablaDT_cell_edit
19        print(c("Valor modificado:", celda))
20      })
```

- Para exportación, utilizaremos el parámetro 'options' de la función 'datatable()', donde indicamos que botones queremos utilizar y el parámetro 'extensions' para indicar que debe crear botones:

```
30    ## Con Descarga de datos
31    output$id_tablaDT <- DT::renderDT(
32                              datatable(datos,
33                                        extensions = "Buttons",
34                                        options = list(
35                                                       dom = 'Bfrtip',  # muestra botones
36                                                       buttons = c('copy', 'csv', 'excel', 'pdf', 'print')
37                                                      )
38                                       )
39                              )
```

6. Visualización de datos

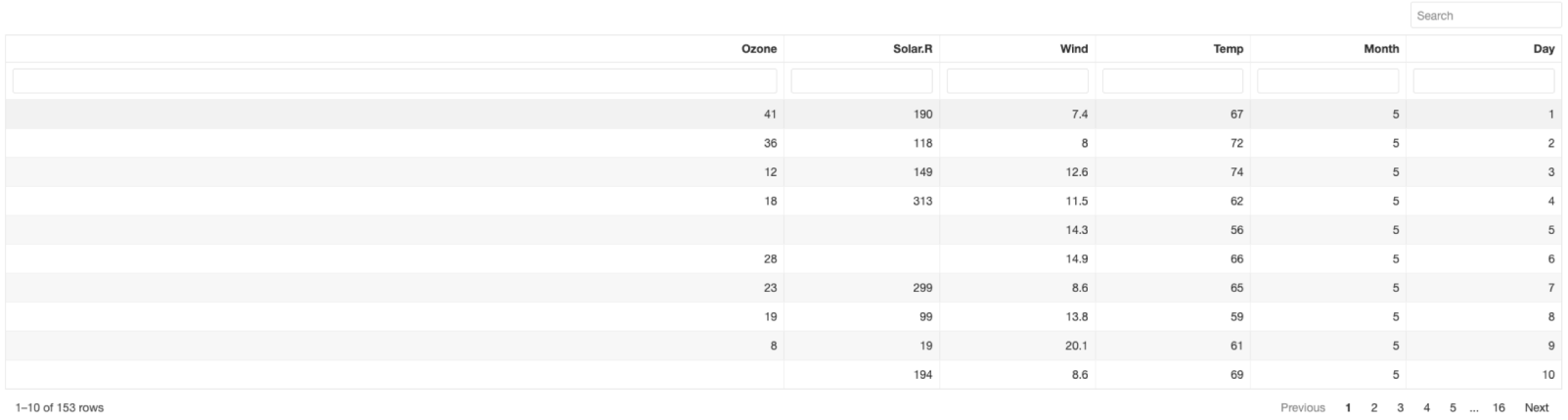
- Reactable (https://glin.github.io/reactable/index.html) funcionalidades muy similares a DT:
- Búsqueda
- Ordenación
- Paginación
- Filtros por columna
- Agrupación (DT también lo permite, pero mucho más complicado y sin ser 'colapsable')
- NO permite edición de celdas
- NO tiene integrada exportación (csv. xlsx…) habría que implementarlo manualmente con downloadButton
- Más personalizable. Estas son algunas de las posibles personalizaciones:
  - Totales

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
		14.3	56	5	5
Total		**1523.50**			

1–5 of 153 rows Previous 1 2 3 4 5 … 31 Next

6. Visualización de datos

- Estilos (personalización de aspecto de la tabla)



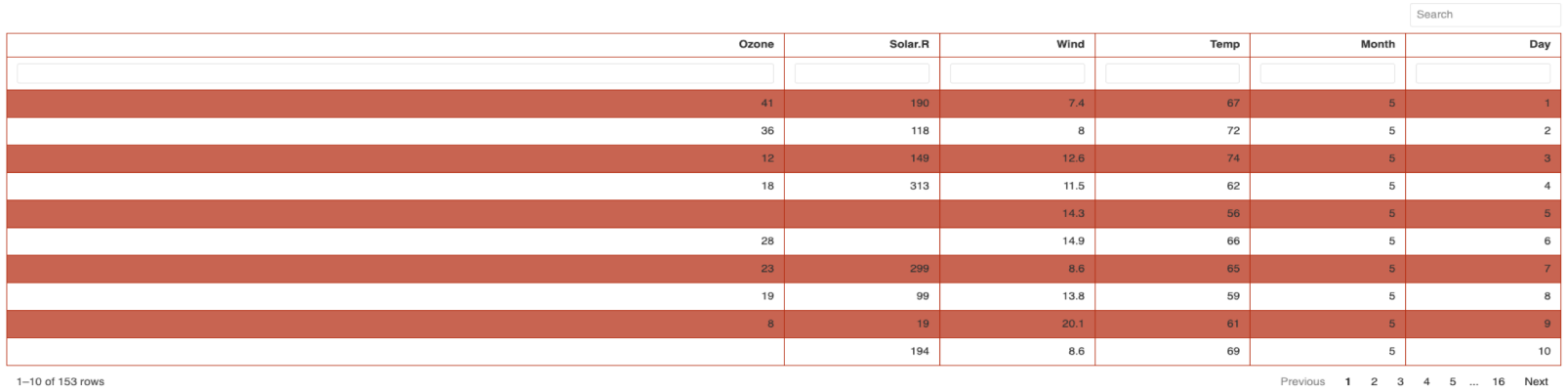
Search

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
		14.3	56	5	5
28		14.9	66	5	6
23	299	8.6	65	5	7
19	99	13.8	59	5	8
8	19	20.1	61	5	9
	194	8.6	69	5	10

1–10 of 153 rows

Previous 1 2 3 4 5 … 16 Next

- Temas (mayor nivel de personalización)



Search

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
		14.3	56	5	5
28		14.9	66	5	6
23	299	8.6	65	5	7
19	99	13.8	59	5	8
8	19	20.1	61	5	9
	194	8.6	69	5	10

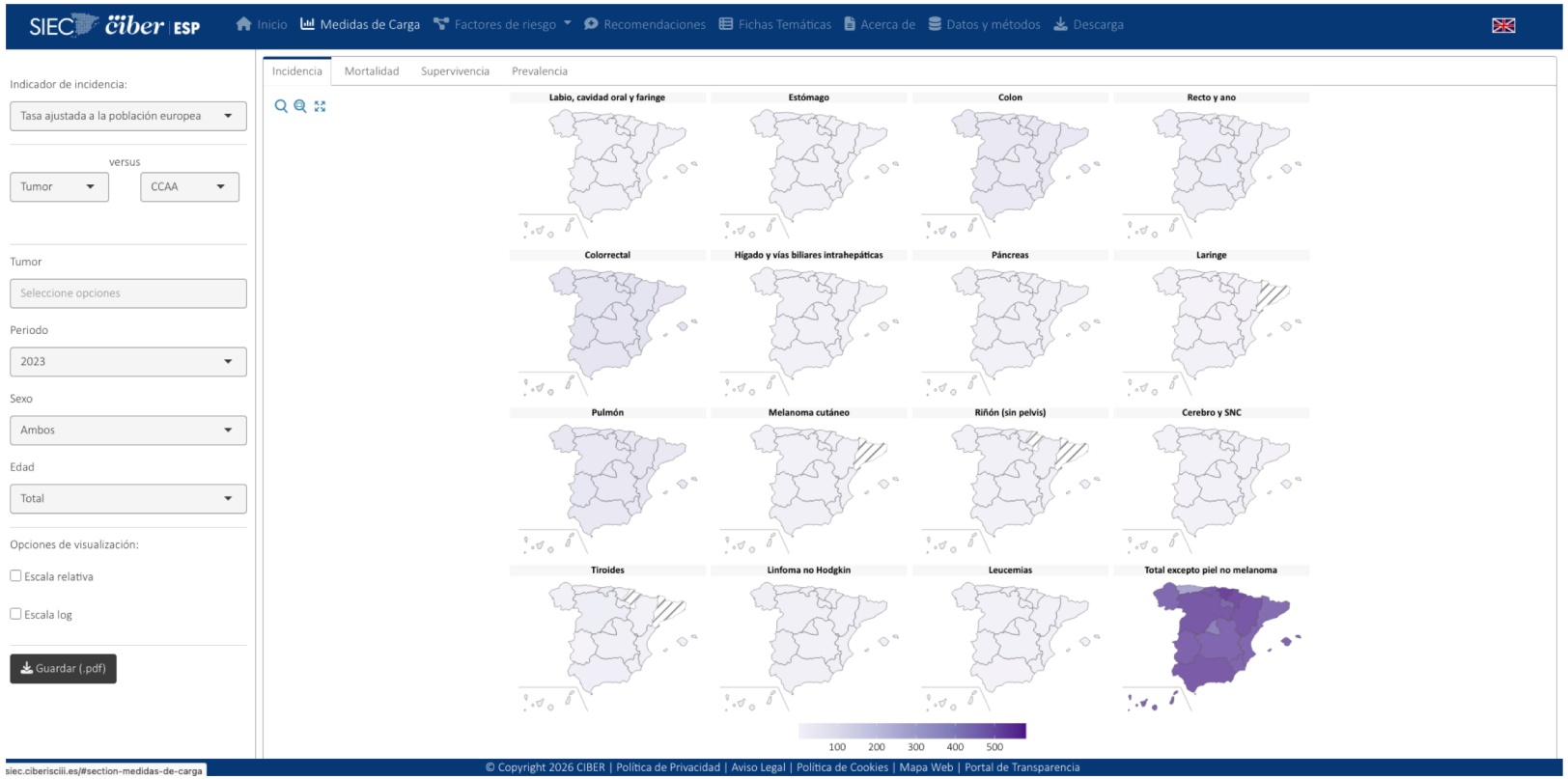
1–10 of 153 rows

Previous 1 2 3 4 5 … 16 Next

7. Creación de dashboards y navegación

7.1 Creación de dashboards.

- El termino 'dashboard' se refiere a panel de control, habitualmente en un lateral de la pantalla, que nos permite incluir controles sobre nuestra aplicación.



7. Creación de dashboards y navegación

- Divide la pantalla en tres secciones:

```
1  library(shiny)
2  library(shinydashboard)
3
4  ### Parte cliente
5  ui <- dashboardPage(
6    dashboardHeader(),
7    dashboardSidebar(),
8    dashboardBody()
9  )
```

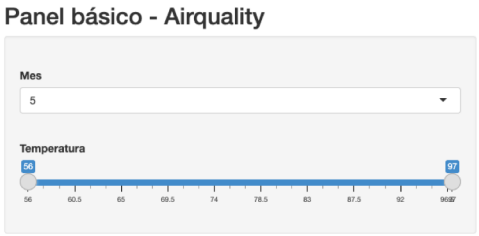
- Cabecera: podemos incluir el ‘titulo’ y los ‘logos’ y controles generales de la aplicación (por ej. Cambio de idioma). Además, ofrece controles para gestión de notificaciones.
- Barra de navegación: donde incluir botones de control de los parámetros para personalizar la salida principal.
- Cuerpo: salida principal de la aplicación. Normalmente tabla con datos, gráficos, etc.

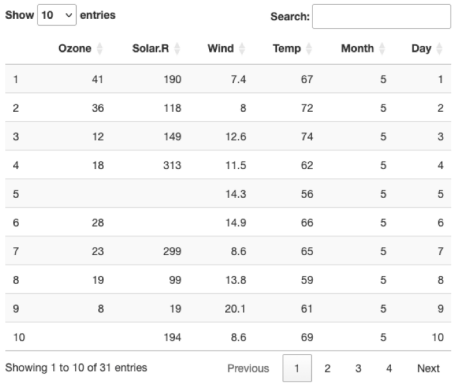
7. Creación de dashboards y navegación

7.2 Estructura de paneles, menús y pestañas.

Otra manera de organizar y subdividir el contenido de nuestra aplicación, es mediante paneles.

- Paneles básicos en shiny: nos ofrece una funcionalidad similar a 'shinydashboard', pero más básica y limitada.

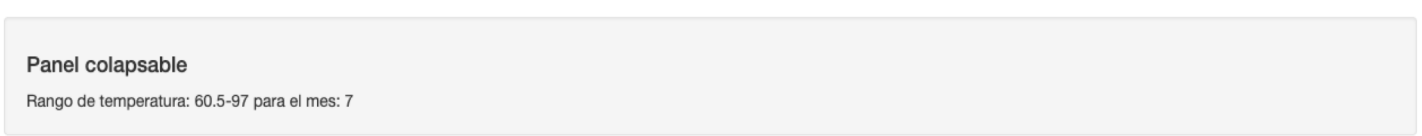




Dispone de una sección 'sidebarPanel', donde incluiremos los controles de parametrización de los datos, y una sección 'mainPanel' donde pondremos el resultado a visualizar (tablas, gráficos, etc.).

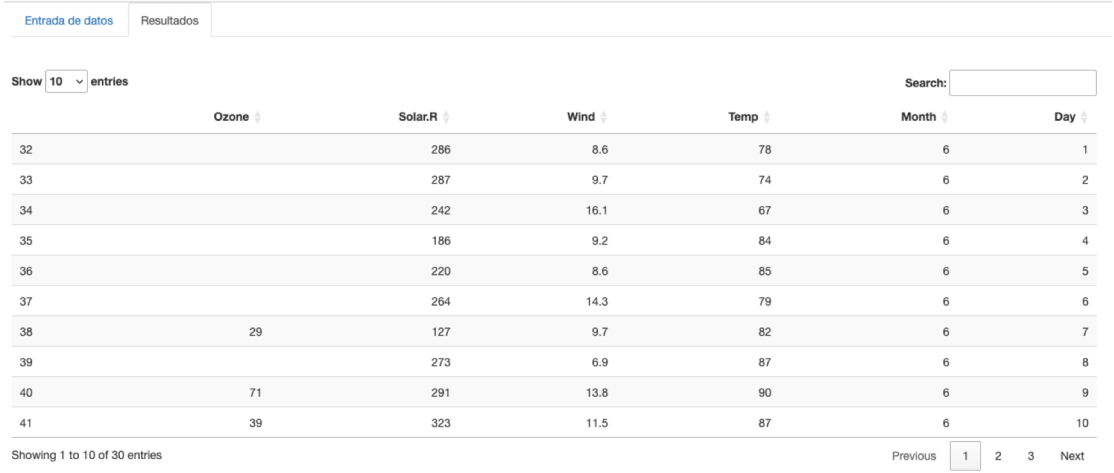
7. Creación de dashboards y navegación

- Paneles visuales: sirven para agrupar elementos dentro de un mismo estilo visual (subdivisión con fondo gris y bordes redondeados).



- Paneles de pestañas: Nos permiten hacer subdivisiones en pestañas, para agrupar el contenido de la aplicación.





7. Creación de dashboards y navegación

- Los 'Paneles de pestañas' (*tabsetPanel*), tienen un parámetro '*id*' que nos permiten referenciarlo en el 'server.R', sin embargo, cada una de las pestañas (*tabPanel*), no son referenciables directamente puesto que no disponen del parámetro '*id*'.

```
# tabsetPanel si tiene 'id'
observeEvent(input$id_tabset,{

  if (input$id_tabset=="Entrada de datos") {
    print(c("Panel seleccionado:", input$id_tabset))
  }
  if (input$id_tabset=="Resultados") {
    print(c("Panel seleccionado:", input$id_tabset))
  }

})
```

7. Creación de dashboards y navegación

7.3 Navegación en varias pantallas.

Además de las facilidades que nos ofrecen los distintos paquetes de shiny, podemos necesitar una aplicación con varias pantallas, en las que necesitemos navegar de unas a otras.

La forma más simple, es definir el cliente (ui) como contenido dinámico (renderizable) y desde el servidor actualizarlo según corresponda.

```
3  shinyUI(
4    (htmlOutput("id_app"))
5  )
```

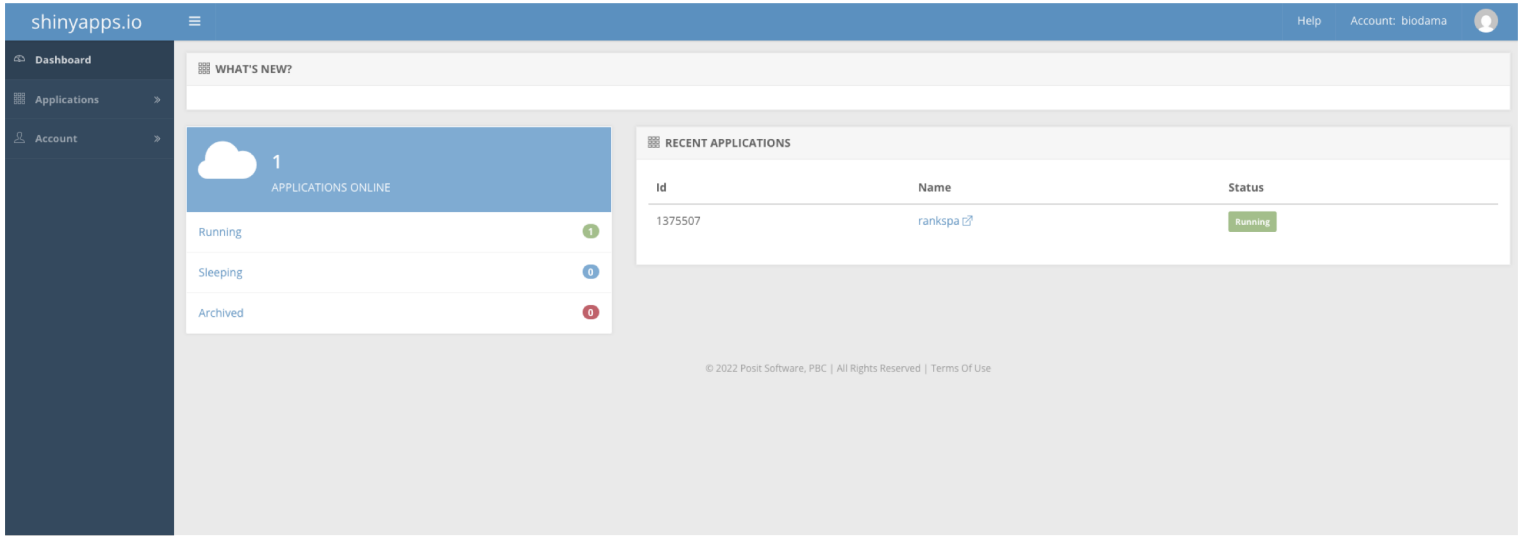
```
8      output$id_app <- renderUI({
9        bootstrapPage(ui_pantalla1())
10     })
```

8. Publicación y despliegue de una aplicación.

Hasta ahora, hemos creado aplicaciones en local, mediante RStudio que simula el funcionamiento de un servidor web, pero que no son accesible fuera de nuestro ordenador. Veremos ahora, como publicar nuestras aplicaciones, para otros usuarios, además de nosotros mismos.

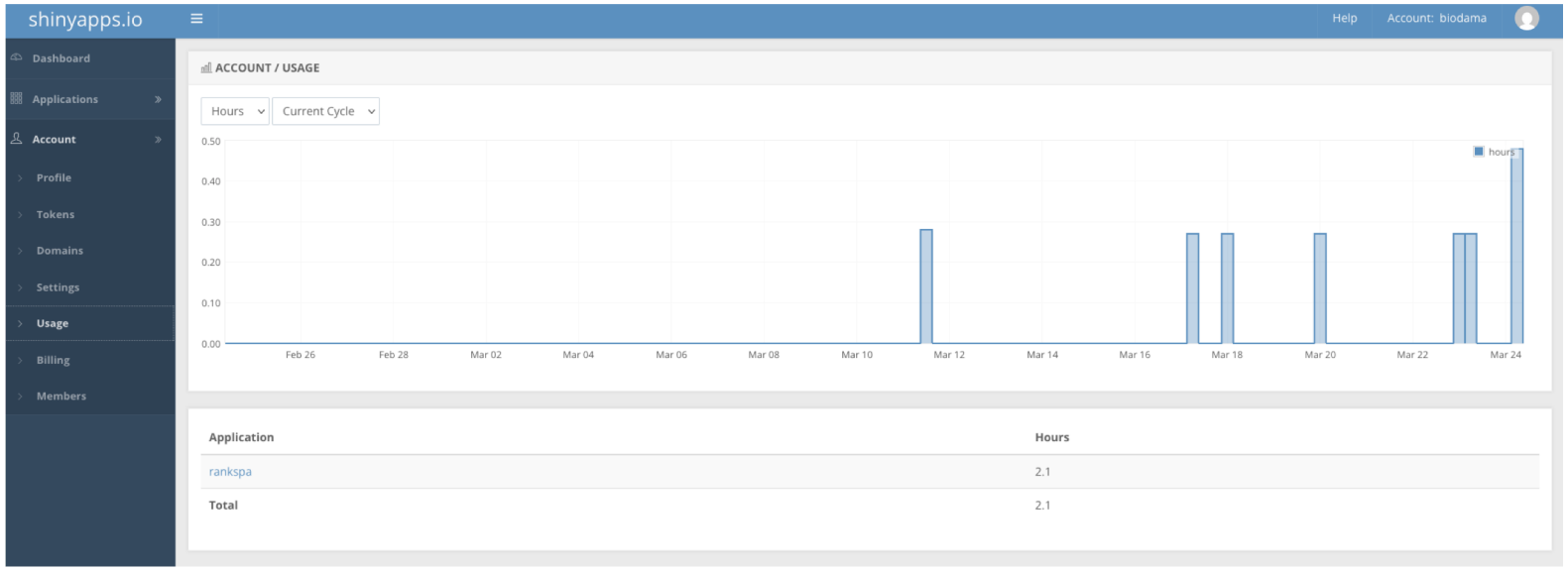
8.1 Servidor externo (shinyapps.io).

- Es un servicio en la nube, que nos permite publicar en internet (accesible por cualquiera) nuestra aplicación de R Shiny. Buena opción, para pequeñas aplicaciones sin gran número de usuarios y sin necesidad de mantenimiento de un servidor web.



8. Publicación y despliegue de una aplicación.

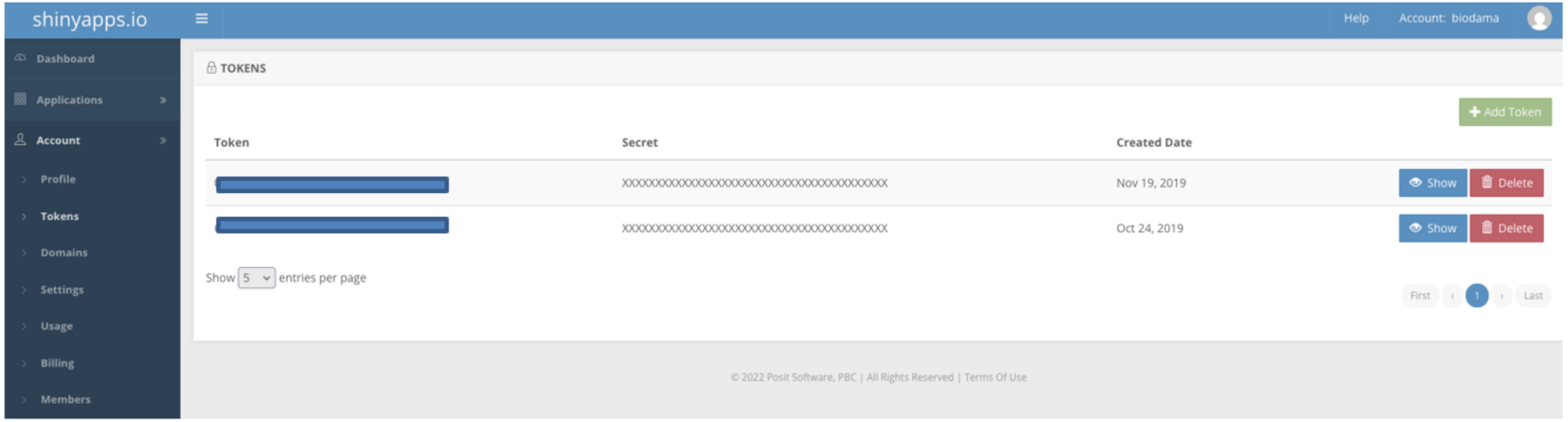
Requiere que nos registremos, y tiene suscripción de pago que nos da más funcionalidades, como por ejemplo 'autenticación' en nuestra aplicación, más horas de servicio al mes y mayor rendimiento. En el registro básico, nos permite tener varias aplicaciones, además nos da estadísticas de uso de nuestra aplicación.



8. Publicación y despliegue de una aplicación.

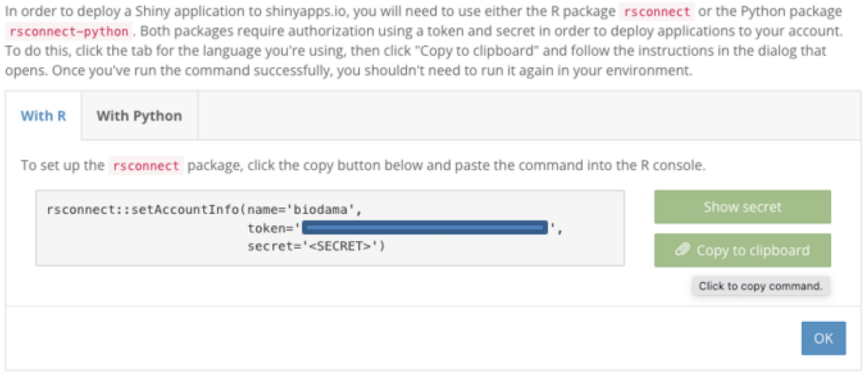
- Para publicar una aplicación, usaremos un script de R:

```
install.packages('rsconnect') # paquete para conexión con shinyapps.io
# Para acceder a tu cuenta, necesitas un 'token' que generas desde tu cuenta
# El secret, está asociado al 'token'
rsconnect::setAccountInfo(name='biodama',
                          token='XXXXXXXXXXXXXXXXXX',
                          secret='xxxxxxxxxxxxxxxxxxxxxxxxx')
```



8. Publicación y despliegue de una aplicación.

- Podemos ver el ‘secret’



Para desplegar la aplicación en el servidor (instalarla o actualizarla):
library(rsconnect)
accounts(server = NULL)
rsconnect::deployApp('/Users/tatum/Desktop/rankspa/',account='biodama')
#Borrar una app:
Primero hay que pararla (aunque esté en estado 'sleeping'), para que te deje borrarla
rsconnect::terminateApp(appName = 'rankspa’)
rsconnect::purgeApp(appName='rankspa') # Segundo: borrado

8. Publicación y despliegue de una aplicación.

8.2 Servidor propio (shiny-server).

Podemos crear nuestro propio servidor web shiny-server, que nos permite publicar varias apliaciones shiny. Esta pensado para sistemas operativos Linux, por lo que en Windows habría que utilizar un contenedor Docker con un sistema Linux instalado.

1. Instalación de R en la máquina que vayamos a utilizar. Con los paquetes necesarios para las aplicaciones a publicar.
2. Instalación del servidor web, shiny-server.
3. Configuración de dicho servidor, mediante el fichero de configuración:
   /etc/shiny-server/shiny-server.conf
4. Inicio del servicio en el sistema operativo, asociado a dicho servidor web:
   systemctl start shiny-server

Con estos pasos, dentro de una red privada, podríamos publicar aplicaciones R Shiny, accesibles solamente en la propia red.

8. Publicación y despliegue de una aplicación.

Para publicar aplicaciones en internet, necesitamos un *dominio* (url) y sería necesario otra capa adicional de seguridad y gestión de acceso y multiples conexiones (normalmente con nginx). En estos casos, podemos contratar servicios de 'hosting' que tengan soporte para aplicaciones shiny, como 'Posit Connect'.

8.3 Como aplicación de escritorio.

Podemos hacer que nuestra aplicación funcione como una aplicación de escritorio, ejecutándose directamente en el navegador, sin necesidad de utilizar Rstudio:

1- Creamos script de R que lanza nuestra aplicación

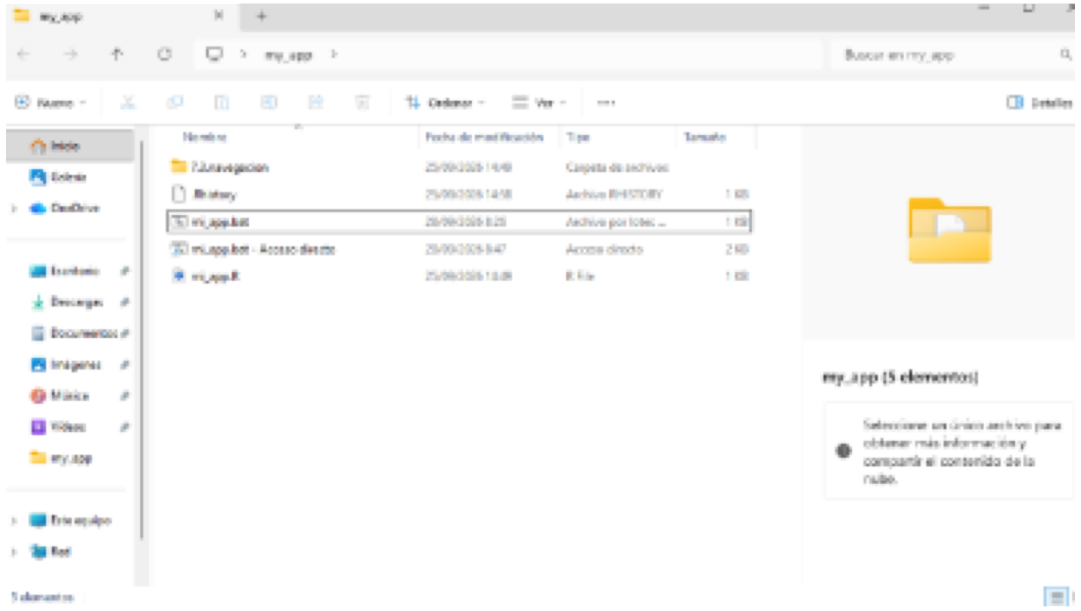
```
library('shiny')
runApp('/Volumes/LaCie/JGP/PC-ISCIII-2/jgp/Personal/FORMACION/Cursos impartidos/2026/Introducion a R Shiny/temario/7.3.navegacion',
       launch.browser = TRUE )
```

2- Creamos script de sistema que lanza el script de R generado anteriormente

```
#!/bin/bash
Rscript --vanilla mi_app.R
```

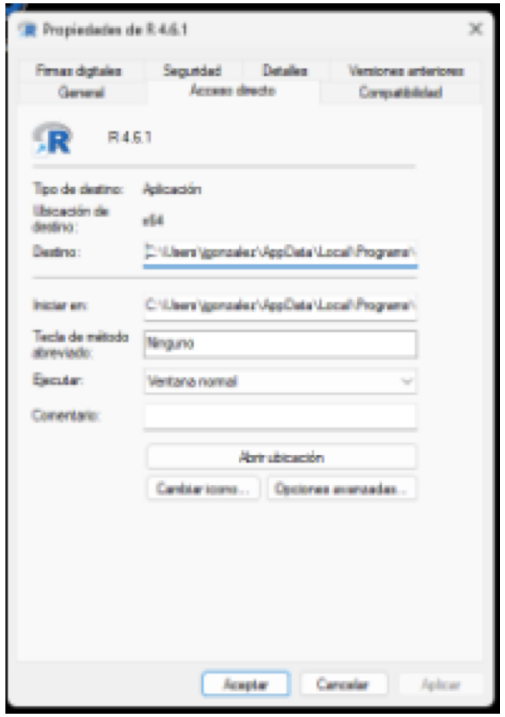
8. Publicación y despliegue de una aplicación.

- *En Windows*:
Partiendo de mi carpeta



1. Localizamos la ruta de instalación de R en el equipo

8. Publicación y despliegue de una aplicación.



2. Creamos un .bat, que mediante Rscript ejecute el script R que contiene la llamada a la app de R Shiny

```
C:\Users\jgonzalez\AppData\Local\Programs\R\R-4.6.1\bin\x64\Rscript --vanilla mi_app.R
```

8. Publicación y despliegue de una aplicación.

3. En el explorador:
    - Botón derecho, ‘Mostrar más opciones’
    - Crear acceso directo

9. Práctica final.

Implementar una aplicación R Shiny, partiendo de los datos datasets::Puromycin de R (base de datos con la velocidad de reacción en función de la concentración del sustrato en una reacción enzimática en la que participan células sin tratar o células tratadas con puromicina.), que cumpla:

1. Contenga selección para las variables:
   - Estado → radio buttons
   - Concentración → combo box (select)
   - Velocidad de reacción → rango de valores (slider)

2. Muestre los datos seleccionados mediante los controles implementados en el punto 1.
3. Permita descargar los datos del punto 2, tanto en formato '.csv' como en '.xlsx'.
4. (Punto adicional). Muestre un gráfico descriptivo (a elegir) de los valores seleccionados.

El contenido de la aplicación se mostrará como se estime oportuno, utilizando cualquier herramienta de las explicadas (dashboards, paneles …).